

Bilkent University, CS 473/573, Session 2, 50pts, 45min

Question 2: Imagine a climber trying to climb on top of a wall. A wall is constructed out of square blocks of equal size, each of which provides one handhold. Some handholds are more dangerous than others. Initially (while on the ground), the climber can start from any one of the blocks in the first row. Once off the ground, from each block the climber is currently on, s/he can reach three blocks of the row just above: the one directly above, the one towards upper right, and the one towards upper left (unless right or left are not available because it is the end of the wall). The goal is to find the least dangerous path from the bottom of the wall to the top, where the danger rating (cost) of a path is the sum of danger ratings (costs) of blocks used on that path.

The problem is presented as follows. The input is an $m \times n$ grid, given as a 2D array $D = (d_{ij})$, where each block has a positive cost d_{ij} associated with it, where the higher the cost, the more dangerous the block. The index of bottom row is **one** and the index of top row is m . From block (i, j) , $i < m$, you can reach blocks

- $(i + 1, j - 1)$ if $j > 1$,
- $(i + 1, j)$, or
- $(i + 1, j + 1)$ if $j < n$

Below is an example of an input block grid, where $m = 4$ and $n = 5$. A least dangerous path is highlighted, which has a total cost of **12**:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

Towards a **dynamic programming algorithm** to find the cost of the least dangerous path:

- a. Give the **recursive definition** for the value of an optimal solution (**Step 2** of dynamic programming formulation).
- b. Write the pseudo-code for the bottom-up algorithm that computes the **cost** of the least dangerous path (**Step 3** of dynamic programming formulation). Memoization-based solutions will be heavily penalized.